aren't as straightforward when we get to configuring system specs with Capybara for end-to-end browser testing.

Keep in mind our focus here is not on how to test your code—I'm assuming you're already bringing that knowledge and experience to the table (and if you aren't, there are plenty of good books on the subject[1]). We're just interested in getting some common testing tools set up in a Dockerized environment.

Once more into the breach…

Setting Up RSpec

Now that our app is configured correctly with Compose, setting up RSpec is going to be very familiar. Let's whiz through this quickly.

Following the instructions from rspec-rails,[2] we need to add the following to our Gemfile:

```
group :development, :test do
  # Call 'byebug' anywhere in the code to stop execution and get a debugger…
  gem 'byebug', platforms: [:mri, :mingw, :x64_mingw]
➤ gem 'rspec-rails', '~> 3.8'
end
```

First, let's stop our web service:

```
$ docker-compose stop web
```

Next, we need to rebuild our image to run bundle install, and then create a new container from it:

```
$ docker-compose build web
Building web
Step 1/12 : FROM ruby:2.6
«...»
Step 7/12 : RUN apt-get update -yqq && apt-get install -yqq --no-install-
recommends   nodejs   yarn
«...»
Bundle complete! 18 Gemfile dependencies, 77 gems now installed.
Bundled gems are installed into `/usr/local/bundle`
«...»
Removing intermediate container dcb3ac9ef4e5
 ---> a1bf00e74754
Step 11/12 : COPY . /usr/src/app/
 ---> 395cd4848b46
Step 12/12 : CMD ["bin/rails", "s", "-b", "0.0.0.0"]
 ---> Running in 47ec46df6236
```

1. https://pragprog.com/book/rspec3/effective-testing-with-rspec-3
2. https://github.com/rspec/rspec-rails